



Royaume du Maroc
L'Office de la Formation
Professionnelle et de
Promotion du Travail (OFPPT)



RAPPORT DE PROJET DE FIN D'ETUDE :

GESTION DES PAIEMENTS



Présente par : Insafe EL GUERGAR

Encadre par : ABDELLATIF SOKLABI

Date de soutenance : 16 juin 2026

REMERCIEMENTS

Je tiens tout d'abord à remercier Allah pour Sa bénédiction, Sa guidance et la force qu'Il m'a accordée tout au long de la réalisation de ce projet.

J'adresse également mes sincères remerciements à mon professeur et encadrant M. ABDELLATIF SOKLABI pour ses conseils et son accompagnement.

Je remercie profondément mes parents pour leur soutien, leurs encouragements et leurs sacrifices.

Enfin, mes remerciements vont à mes camarades de classe pour leur aide, leur esprit de collaboration et les moments partagés durant cette formation.

RÉSUMÉ

Le présent rapport décrit la conception, le développement et la validation d'une plateforme de gestion de paiements nommée « Gestion des Paiements » . Il s'agit d'une application web full-stack écrite exclusivement en PHP natif, suivant le patron MVC et reposant sur PDO pour l'accès sécurisé à une base de données MySQL.

Le système intègre une interface unifiée de fournisseurs de paiement : Stripe est pleinement opérationnel en mode test, tandis que PayPal sont prévus dans l'interface utilisateur et retournent un message « bientôt disponible » lorsqu'ils ne sont pas encore configurés.

Un accent particulier a été mis sur la sécurité (authentification JWT sans état, requêtes préparées PDO, limitation de débit) et sur l'expérience utilisateur grâce à une interface responsive et un parcours de paiement clair (sélection du plan, récapitulatif avec TVA, saisie carte via Stripe Elements).

L'ensemble du code a été élaboré sans recours à des Framework lourds, garantissant une empreinte minimale et une aisance de déploiement dans des environnements d'hébergement partagés (XAMPP / Apache / MySQL). Les résultats montrent que la plateforme répond aux exigences fonctionnelles définies au départ, tout en restant extensible (finalisation des autres fournisseurs, génération de factures PDF, containerisation Docker).

TABLE DES MATIÈRES

Remerciements.....	2
Resume	3
Error! Bookmark not defined. Table des matieres.....	4
I. Introduction	6
1. Contexte	6
2. Problematique.....	6
3. Objectifs du projet	6
4. Methodologie de travail	7
II. Presentation du projet	8
1. Description du projet	8
2. Contexte du projet	8
3. Objectif general du projet	8
4. Fonctionnalites principales de la plateforme.....	8
5. Public cible.....	8
6. Conclusion.....	9
III. Analyse des besoins.....	10
1. Expression du besoin	10
a Besoins fonctionnels.....	10
b Besoins non fonctionnels.....	10
3. Contraintes du projet	11
4. Conclusion	11
IV. Conception.....	12
1. Architecture generale du systeme.....	12
2. Diagramme de cas d'utilisation.....	12
3. Conception fonctionnelle	13
4. Conception de la base de donnees	13
5. Conception de l'interface utilisateur	14
6. Securite et gestion des acces	16
7. Conclusion.....	17
V. Technologies utilisees	18
1. PHP.....	18
2. MySQL	18
3. HTML	18
4. CSS	18
5. JavaScript	18

6.Github.....	18
7.Figma.....	19
8. Conclusion.....	19
VI. Developpement de l'application.....	20
1. Introduction	20
2. Environnement de developpement	20
3. Organisation de l'application.....	20
Front-end	20
Back-end.....	20
4. Developpement du Front-end.....	20
5. Pages principales developpees	21
6. Developpement du Back-end	21
7. Securite et gestion des acces	21
8. Conclusion.....	22
VII. Tests et validation	23
1. Introduction	23
2. Objectifs des tests	23
3. Tests fonctionnels	23
4. Tests de validation des formulaires	23
5. Tests de la base de donnees.....	24
6. Tests d'interface utilisateur.....	24
7. Resultats.....	24
Anomalies identifiees et corrigees	25
8. Validation du systeme.....	25
9. Conclusion.....	25
VIII. Resultats et discussion	26
1. Introduction	26
2. Resultats obtenus	26
3. Comparaison avec les objectifs initiaux	26
4. Apports du projet	26
5. Difficultes rencontrees	27
6. Discussion et limites du projet	27
7. Conclusion.....	27
IX. Conclusion generale et perspectives	28
1. Conclusion generale.....	28
2. Perspectives d'evolution.....	28
X. Bibliographie.....	29

I. Introduction

1. Contexte

A l'ère de la transformation numérique, la dématérialisation des flux financiers est devenue un enjeu stratégique majeur pour les entreprises, les administrations et les plateformes de commerce électronique. La mise en place d'un système de gestion de paiements efficace ne se limite plus à la simple exécution de transactions, mais englobe la fluidité de l'expérience utilisateur, l'automatisation des processus et, par-dessus tout, la sécurité des données sensibles.

L'idée de départ est de concevoir une plateforme de paiement capable de :

- Accepter des transactions ponctuelles et des abonnements (mensuels ou annuels) via plusieurs fournisseurs ;
- Suivre le cycle de vie complète d'une transaction (status pending, completed, failed, refunded, partially refunded) ;
- Fournir une API RESTful sécurisée pour l'intégration avec des frontends (web, mobile)
- Assurer la traçabilité (journaux de paiement, historiques, audit).

2. Problématique

Comment concevoir une architecture logicielle PHP capable d'intégrer de manière homogène plusieurs passerelles de paiement hétérogènes, tout en minimisant la latence des transactions et en assurant une protection efficace contre la fraude et les fuites de données ?

3. Objectifs du projet

- **Gestion complète du cycle de paiement** : enregistrement, validation, traitement et suivi des transactions (statuts pending, completed, failed, refunded).
- **Séquestre du montant** : le champ `net_amount` représente le montant retenu jusqu'à confirmation ; les remboursements sont traités via la fonction `refundPayment` du fournisseur.
- **Facturation** : génération de factures (invoices) à partir des transactions via `InvoiceModel` et `InvoiceService`.
- **Administration** : tableau de bord avec statistiques financières, liste des transactions, gestion des utilisateurs, des plans d'abonnement, des coupons et des journaux applicatifs.

- **Self-service client** : API RESTful pour les utilisateurs (consultation des transactions, abonnements, factures, gestion du compte).
- **Intégration de plusieurs fournisseurs de paiement** via une interface unifiée (PaymentProviderInterface et ProviderFactory) : Stripe, PayPal Les fournisseurs non configurés renvoient explicitement une RuntimeException avec le message « This payment method is coming soon. Please use Stripe for now. »
- **Securite**: authentification JWT (JWTService), controle d'accès via AuthMiddleware, limitation de débit (RateLimitMiddleware).
- **Interface utilisateur cohérente** : vues PHP avec assets corrigés et navigation dynamique côté client (objet BASE en JavaScript).
- **Qualité du code et maintenabilité** : architecture MVC, services dédiés, modèles encapsulant la persistance, fonctions utilitaires (db(), successResponse(), errorResponse()).

4. Méthodologie de travail

Afin de mener à bien ce projet, une méthodologie de travail structurée a été adoptée :

1. **Analyse des besoins** : identification des fonctionnalités et des entités métier (utilisateurs, plans, abonnements, transactions, coupons, factures).
2. **Conception** : modélisation du système (diagramme de cas d'utilisation) et conception de la base de données relationnelle.
3. **Développement** : implémentation des fonctionnalités selon une architecture MVC en PHP natif (contrôleurs, services, modèles).
4. **Tests et validation** : vérification du bon fonctionnement de l'application (authentification, parcours de paiement, tableau de bord) et correction des anomalies identifiées via les journaux PHP.
5. **Documentation** : rédaction du présent rapport et mise à jour de la configuration (.env, .htaccess).

II. Présentation du projet

1. Description du projet

Gestion des Paiements est une application web permettant de gérer l'ensemble du cycle de vie d'un paiement : choix d'un plan, application d'un coupon, paiement via un fournisseur, suivi du statut, remboursement et facturation. L'application expose une API RESTful consommée par des vues PHP via un client JavaScript unique (assets/js/api.js).

2. Contexte du projet

Le projet a été réalisé dans le cadre du projet de fin d'étude de la formation Technicien Spécialisé en Développement Digital (OFPPT). Il s'agit d'un projet individuel, développé en environnement local (XAMPP) avec une base de données MySQL nommée paymentmodule.

3. Objectif général du projet

Fournir une plateforme de gestion de paiements robuste, sécurisée et évolutive, structurée selon une architecture MVC en PHP natif, capable d'unifier plusieurs canaux de paiement derrière une interface commune.

4. Fonctionnalités principales de la plateforme

- Inscription, connexion et gestion de session via JWT (avec jeton de rafraîchissement).
- Consultation des plans d'abonnement et souscription (mensuelle ou annuelle).
- Paiement via Stripe (Credit Card) avec saisie sécurisée de la carte via Stripe Elements ; PayPal, sont présents dans l'interface mais affichent un statut « bientôt disponible ».
- Application de codes de réduction (coupons) lors du paiement.
- Suivi des transactions, abonnements et factures par l'utilisateur.
- Tableau de bord administrateur : statistiques, transactions, utilisateurs, abonnements, plans, coupons, remboursements et journaux d'activité.

5. Public cible

- Utilisateurs finaux souhaitant souscrire à un plan ou effectuer un paiement ponctuel ;
- Administrateurs assurant la supervision financière de la plateforme ;
- Équipes techniques amenées à intégrer de nouveaux fournisseurs de paiement.

6. Conclusion

Cette presentation generale met en evidence un projet centre sur la gestion securisee de paiements en ligne, avec une architecture pensee pour evoluer progressivement vers le support complet de plusieurs fournisseurs de paiement.

III. Analyse des besoins

1. Expression du besoin

a. Besoins fonctionnels

Besoin	Implementation observe
Gestion des utilisateurs	UserModel (create, findByUuid, count, update). Authentification via JWT (JWTService).
Gestion des plans abonnement	PlanModel et CRUD complet dans AdminController (listPlans, createPlan, updatePlan, deletePlan).
Gestion des coupons	Tables coupons / coupon_usages, CouponService pour la validation lors du paiement, CRUD dans AdminController.
Traitement des transactions	TransactionModel : amount, tax_amount, discount_amount, net_amount, currency, status, provider, informations de facturation.
Sequestre du paiement	Le champ net_amount represente le montant retenu jusqu'à confirmation du service ; les remboursements utilisent refundPayment() du fournisseur.
Facturation	InvoiceModel et InvoiceService generent les factures à partir des transactions completees.
Tableau de bord administrateur	AdminController::dashboard agrege chiffre d'affaires, transactions recentes, plans les plus vendus.
Securite des API	AuthMiddleware verifie le jeton JWT et le role (admin ou user) sur chaque route protegee.
Support multi-fournisseur	ProviderFactory centralise la selection du fournisseur (Stripe, PayPal) ; StripeProvider est operationnel, les autres renvoient une exception explicite.
Journalisation des paiements	Table payment_logs et endpoint GET /api/admin/logs pour l'audit et la tracabilite.

b. Besoins non fonctionnels

- Sécurité : mots de passe hashes (bcrypt), requetes preparees PDO, jetons JWT signes (HMAC-SHA256), limitation de debit (rate limiting).
- Performance : reponses JSON uniformes via successResponse()/errorResponse(), requetes paginees (page, limit) sur les listes (transactions, utilisateurs, abonnements, logs).
- Ergonomie : interface responsive (mise en page adaptative en deux colonnes sur la page de checkout, une colonne sur mobile), retours visuels (toasts, etats de chargement).
- Maintenabilité : separation claire entre controleurs, services et modeles ; ajout d'un nouveau fournisseur de paiement sans modifier les controleurs existants.

- Portabilité : application deployable sur un hébergement partage (PHP + MySQL), sans dépendance à un framework lourd.

2. Constraints du projet

- Développement réalisé en environnement local XAMPP (Apache, PHP, MySQL via phpMyAdmin).
- Délai imparti au projet de fin d'étude, limitant l'intégration complète de tous les fournisseurs de paiement à Stripe pour la version testée.
- Nécessite de configurer manuellement le serveur Apache (.htaccess) pour la transmission de l'en-tête Autorisation, indispensable au fonctionnement du JWT.

3. Conclusion

L'analyse des besoins confirme que les fonctionnalités essentielles (authentification, gestion des plans/abonnements/coupons, paiement, administration, journalisation) sont couvertes par l'architecture mise en place, le périmètre restant à compléter concernant principalement la finalisation des fournisseurs de paiement secondaires.

IV. Conception

1. Architecture générale du système

La phase de conception constitue l'étape permettant de traduire les besoins en une architecture logicielle robuste et sécurisée. L'architecture globale de gestion des paiements s'articule autour de trois composants, organisés selon une architecture web classique à trois niveaux (3-tier), implémentée de manière monolithique en PHP natif selon le patron MVC :

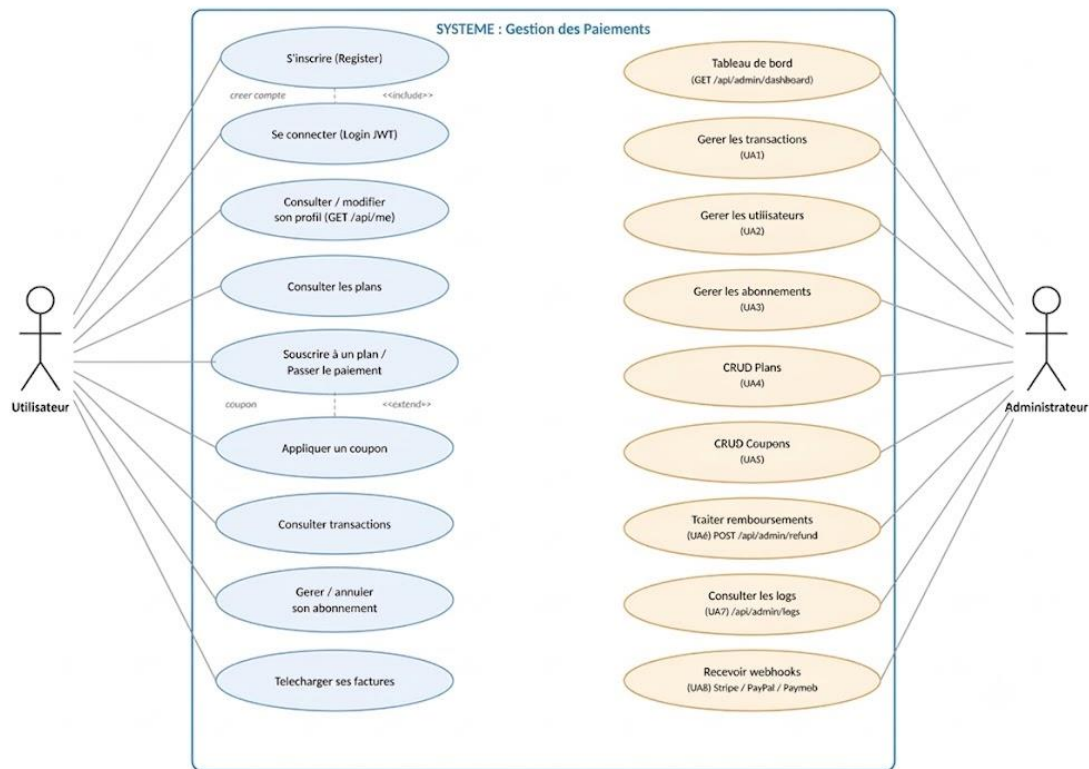
- Le Client (Frontend) : l'interface utilisateur s'exécutant dans le navigateur. Elle émet les requêtes HTTP (sélection de plan, initiation de paiement, consultation de factures) via le module `assets/js/api.js` et gère localement le jeton JWT.
- Le Serveur (Backend) : application structurée selon le patron MVC en PHP natif. Le point d'entrée `public/index.php` instancie le routeur (`app/routes/Router.php`) qui dispatch vers les contrôleurs définis dans `app/routes/api.php` (API) et `app/routes/web.php` (pages).
- La Base de données (Data Store) : moteur relationnel MySQL, accède via PDO (fonction utilitaire `db()`), assurant la persistance et l'intégrité des données sensibles (comptes, abonnements, jetons, journaux d'audit).

2. Diagramme de cas d'utilisation

Afin de représenter les interactions entre les utilisateurs et le système, un diagramme de cas d'utilisation a été réalisé.

Ce diagramme présente les différentes fonctionnalités offertes par le système ainsi que les acteurs impliqués.

Diagramme de cas d'utilisation - Gestion des Paiements

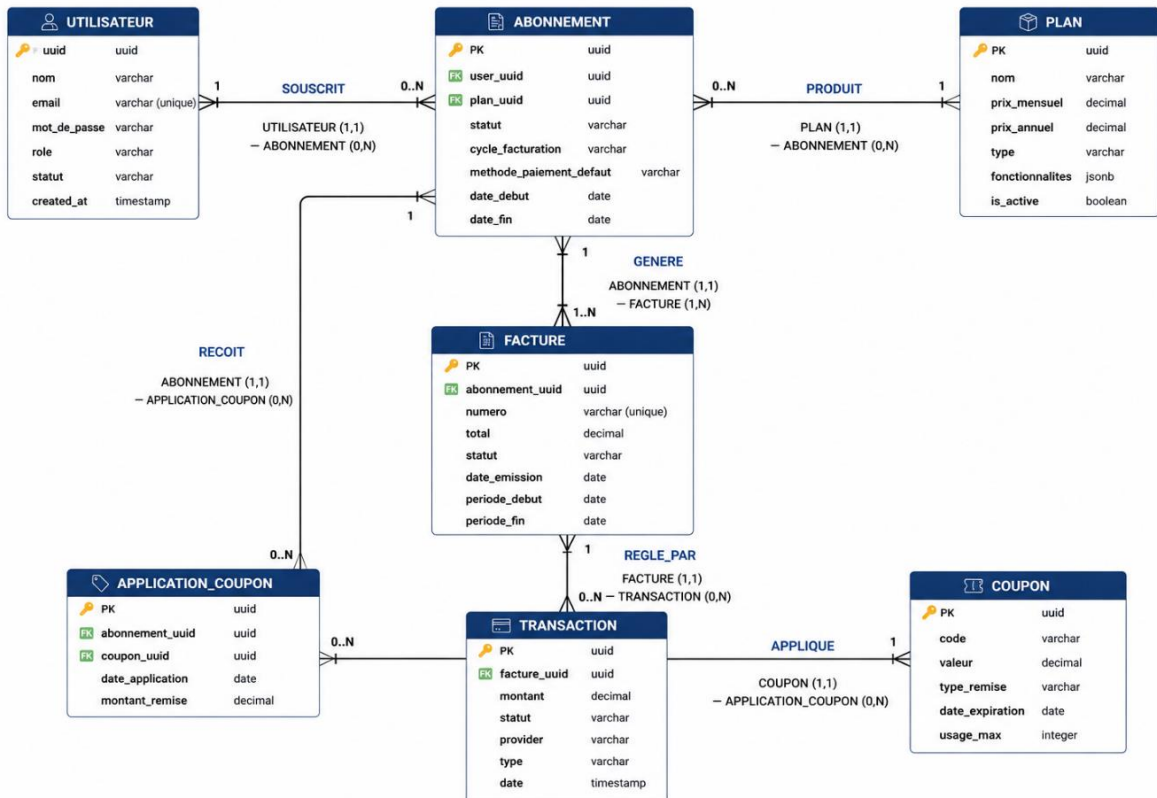


3. Conception fonctionnelle

- Inscription / connexion (JWT) avec jeton d'accès et jeton de rafraîchissement.
- Création / modification / suppression de plans et de coupons (CRUD administrateur).
- Paiement d'un abonnement : création de la transaction, appel du fournisseur via ProviderFactory, mise à jour du statut après vérification.
- Remboursement via l'endpoint administrateur POST /api/admin/refund, avec mise à jour du montant remboursable et du statut de la transaction.
- Génération des factures à partir des transactions complétées (InvoiceService).
- Tableau de bord avec indicateurs clés (revenus, nombre d'utilisateurs, transactions récentes, plans les plus vendus).

4. Conception de la base de données

Description du MCD Le Modèle Conceptuel de Données formalise la structure logique de la base de données paymentmodule. Il isole les entités métier afin de garantir l'atomicité des données et d'éviter toute redondance :



5. Conception de l'interface utilisateur

L'interface utilisateur est composée de vues PHP (dossier views/) accompagnées d'une feuille de style unique (public/assets/css/main.css) fondée sur des variables CSS (couleurs, espacements, rayons de bordure), garantissant une apparence cohérente sur l'ensemble des pages :

- **pricing.php** : présentation des plans d'abonnement ;
- **checkout.php** : sélection du moyen de paiement (Stripe, PayPal), formulaire de carte bancaire (Stripe Elements), récapitulatif de commande avec sous-total, TVA et période d'essai ;
- **auth/login.php, auth/register.php** : authentification
- **user/dashboard.php** : espace personnel (transactions, abonnement, factures) ;
- **admin/dashboard.php** : tableau de bord administrateur (statistiques, transactions, utilisateurs, abonnements, plans, coupons, logs).

La mise en page est responsive : la page de checkout utilise une grille à deux colonnes (formulaire / récapitulatif) qui passe en une seule colonne sur les écrans étroits. Les interactions dynamiques (chargement des données, validation, notifications) sont assurées par le module JavaScript assets/js/api.js.

5 Maquettes des interfaces (Wireframes)

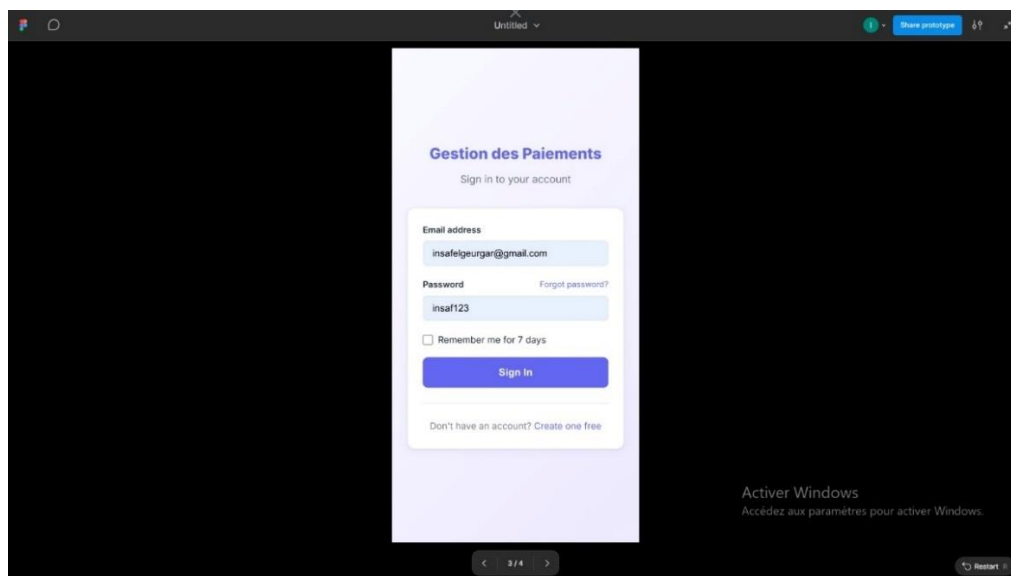


Figure 4.x : Maquette de l'interface d'authentification (auth/login.php) sur appareil mobile.

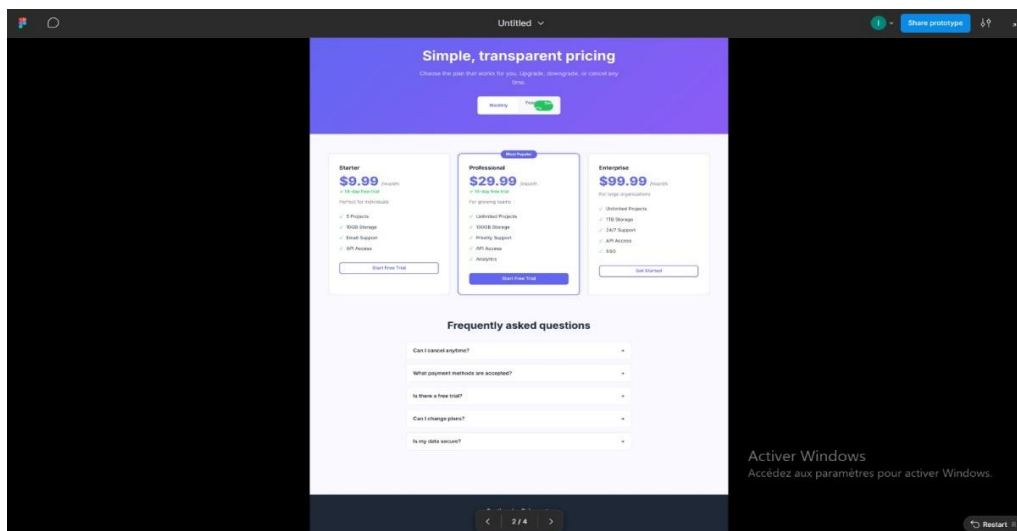


Figure 4.y : Maquette de la page de tarification et présentation des plans (pricing.php).

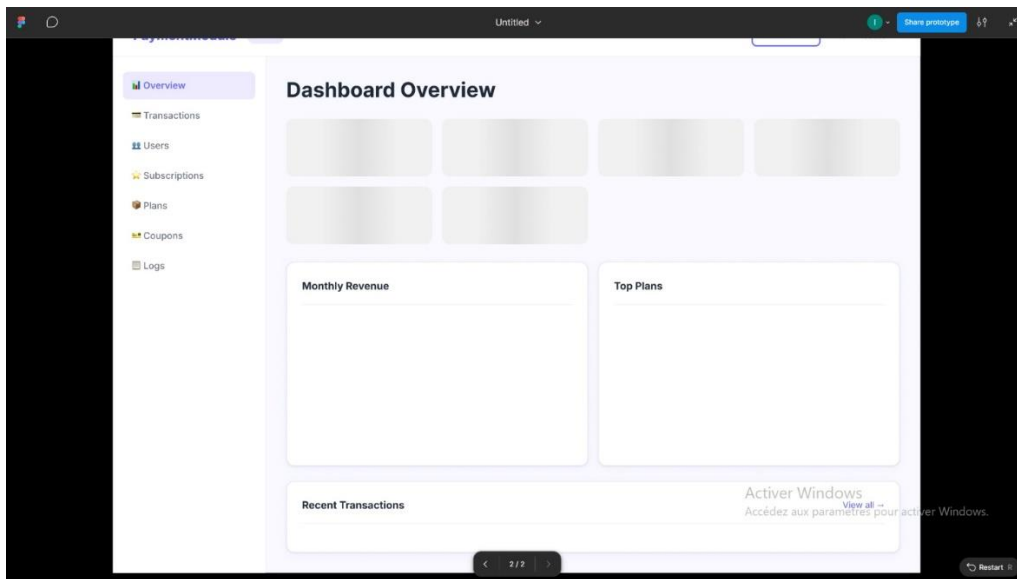


Figure 4.z : Maquette du tableau de bord d'administration (`admin/dashboard.php`) et indicateurs clés.

6. Sécurité et gestion des accès

- Authentification par jetons JWT (JWTService) : génération, signature HMAC-SHA256, vérification de l'expiration, identification de l'utilisateur (champ sub).
- `AuthMiddleware::handle()` contrôle la validité du jeton et le statut du compte ; `AuthMiddleware::requireAdmin()` restreint l'accès aux routes `/api/admin/*` au rôle administrateur.
- Mots de passe stockés avec `password_hash()` (algorithme BCRYPT).
- Requetés préparées PDO sur l'ensemble des modèles pour prévenir les injections SQL.
- `RateLimitMiddleware` limite le nombre de requêtes par clé (ex. connexions, remboursements) en s'appuyant sur la table `rate_limits`.
- Journalisation des événements sensibles dans `payment_logs` et dans `storage/logs/php_errors.log`.
- Configuration du fichier `.htaccess` pour transmettre l'en-tête HTTP Authorization à PHP (règle `RewriteRule`), condition nécessaire au fonctionnement de l'authentification JWT sous Apache/XAMPP.

7. Conclusion

La phase de conception a permis de définir une architecture MVC claire, un schéma de base de données cohérent et une interface utilisateur responsive, tout en intégrant les exigences de sécurité (JWT, PDO, rate limiting) identifiées lors de l'analyse des besoins.

V. Technologies utilisées

1. PHP

PHP (natif, sans framework) constitue le cœur de l'application : routage (app/routes/Router.php), contrôleurs, modèles (accès aux données via PDO) et services (règles métier telles que le calcul de la TVA, la validation des coupons, la génération et la vérification des jetons JWT).

2. MySQL

MySQL est utilisé pour stocker l'ensemble des données du système : utilisateurs, plans, abonnements, transactions, coupons, factures, remboursements et journaux. L'accès se fait exclusivement via PDO et des requêtes préparées.

3. HTML

HTML est utilisé pour structurer les différentes vues de l'application (pages publiques, formulaires d'authentification, tableaux de bord).

4. CSS

Une feuille de style unique (main.css), fondée sur des variables CSS personnalisées, assure la mise en forme et le design responsive de l'ensemble de l'application, sans recours à un framework CSS externe.

5. JavaScript

JavaScript (ES6, vanilla) est utilisé pour la validation des formulaires, les appels asynchrones à l'API (module assets/js/api.js, basé sur fetch), la gestion du jeton JWT (stockage, rafraîchissement automatique en cas de réponse 401) et l'intégration de Stripe.js / Stripe Elements pour la saisie sécurisée des informations de carte bancaire.

6. GitHub

GitHub est utilisé comme plateforme d'hébergement de code et de gestion de versions basée sur Git. Il assure le suivi des modifications du code source, la gestion des branches pour le développement des fonctionnalités (le routage, le module de paiement, l'authentification), et sécurise le code grâce à des sauvegardes distantes régulières.

7. Figma

Figma est l'outil collaboratif de conception graphique vectorielle et de prototypage utilisé pour la création de la charte graphique et des maquettes fonctionnelles (Wireframes) du système. Il a permis de concevoir l'interface utilisateur (UI) et d'optimiser l'expérience utilisateur (UX) pour l'ensemble des pages (tarification, paiement, tableaux de bord) avant l'étape d'intégration en HTML/CSS.

8. Conclusion

L'ensemble de ces technologies, combinées sans framework lourd, permet d'obtenir une application légère, facile à comprendre et à déployer sur un environnement d'hébergement partage de type XAMPP.

VI. Développement de l'application

1. Introduction

Après les phases d'analyse des besoins et de conception, la phase de développement consiste à transformer les modèles et spécifications définis précédemment en une application web fonctionnelle, en utilisant les technologies retenues.

2. Environnement de development

- Maquete :figma
- Serveur local : XAMPP (Apache + MySQL + PHP)
- Editeur de code : Visual Studio Code
- Navigateur de test : Google Chrome
- Base de donnees : MySQL (administree via phpMyAdmin)

Cet environnement permet de simuler le fonctionnement d'un serveur web reel et de deboguer l'application via les journaux PHP (storage/logs/php_errors.log).

3. Organisation de l'application

L'application est organisée de manière à séparer les responsabilités :

Front-end

- Vues PHP (HTML) pour la structure des pages ;
- Feuille de style CSS unique pour le design et le responsive ;
- JavaScript (api.js) pour les interactions client et les appels à l'API.

Back-end

- Contrôleurs PHP pour le traitement des requêtes ;
- Services pour la logique métier (TVA, coupons, JWT, factures) ;
- Modelés pour la persistance via PDO ;
- Middlewares pour l'authentification et la limitation de débit.

4. Développement du Frontend

Le front-end permet aux utilisateurs d'interagir avec l'application :

- Vues PHP pour les pages (accueil/pricing, checkout, tableaux de bord) ;
- CSS personnalise pour un design responsive (grille adaptative, cartes, badges) ;
- JavaScript ES6 pour la validation de formulaires, les appels AJAX (fetch) et la selection du fournisseur de paiement.

5. Pages principales developpees

Page	Role
pricing.php	Presentation des plans d'abonnement disponibles.
checkout.php	Selection du moyen de paiement, saisie de la carte (Stripe Elements), recapitulatif de commande, application de coupon.
auth/login.php / register.php	Connexion et inscription des utilisateurs.
user/dashboard.php	Espace personnel : transactions, abonnement, factures.
admin/dashboard.php	Tableau de bord administrateur : statistiques, transactions, utilisateurs, abonnements, plans, coupons, logs, remboursements.
payment-success.php / payment-failed.php	Pages de retour apres tentative de paiement.

6. Développement du Back-end

Le back-end est développe en PHP selon une architecture MVC :

- Routeur (public/index.php et app/routes/Router.php) : analyse l'URL, instancie le contrôleur approprié et transmet les paramètres.
- Contrôleurs : AuthController (inscription, connexion, rafraichissement de jeton), CheckoutController (initiation et verification du paiement), SubscriptionController (gestion des abonnements), AdminController (administration), PlanController, CouponController, InvoiceController, WebhookController, PageController (rendu des vues).
- Modeles : UserModel, PlanModel, SubscriptionModel, TransactionModel, InvoiceModel, RefundModel, etc., encapsulant les requetes PDO.
- Services : JWTService (jetons), TaxService (TVA), CouponService (validation des reductions), InvoiceService (construction des factures).
- Fournisseurs de paiement : ProviderFactory instancie StripeProvider (operationnel), ou PayPalProvider / PaymobProvider / CMIPProvider qui lèvent une RuntimeException explicite tant qu'ils ne sont pas configurés.

7. Securite et gestion des access

- Hachage des mots de passe avec password_hash() (BCRYPT).
- Authentification par jeton JWT signe (Authorization: Bearer <token>), verifie à chaque requete protegee par AuthMiddleware.
- Prevention des injections SQL grace aux requetes preparees PDO et à la validation des entrees (validateRequired, filter_var).
- Limitation du nombre de requetes sensibles (connexion, inscription, remboursement) via RateLimitMiddleware.

- Reponses JSON uniformisees (successResponse / errorResponse) facilitant la gestion des erreurs cote client.

8. Conclusion

La phase de developpement a permis de mettre en œuvre l'ensemble des fonctionnalites prevues lors de la conception. L'application obtenue est fonctionnelle, interactive et conforme aux objectifs du projet, le parcours de paiement Stripe ayant ete valide de bout en bout en environnement de test.

VII. Tests et validation

1. Introduction

Après la phase de développement, des tests ont été réalisés afin de vérifier le bon fonctionnement de l'application web et de s'assurer qu'elle répond correctement aux besoins définis lors de l'analyse. Cette phase a permis d'identifier des anomalies, de les corriger et de valider les fonctionnalités mises en place.

2. Objectifs des tests

- Vérifier le bon fonctionnement des différentes fonctionnalités de l'application ;
- S'assurer de la cohérence des données stockées dans la base de données ;
- Garantir la stabilité et la fiabilité de l'application ;
- Valider la conformité du système avec les objectifs du projet.

3. Tests fonctionnels

Les tests fonctionnels permettent de vérifier que chaque fonctionnalité de la plateforme fonctionne conformément aux attentes. Les tests réalisés :

- Test de l'inscription et de la connexion des utilisateurs (génération du jeton JWT) ;
- Test du contrôle d'accès : une requête sans jeton vers `/api/admin/dashboard` renvoie une erreur 401 « Unauthenticated. Please log in. » (comportement attendu) ;
- Test du tableau de bord administrateur avec un jeton valide possédant le rôle admin ;
- Test de la consultation des plans d'abonnement et de la souscription (plan « starter », cycle mensuel) ;
- Test de l'application d'un coupon de réduction (cas valide et cas invalide) ;
- Test du paiement via Stripe en mode test (carte 4242 4242 4242 4242).

4. Tests de validation des formulaires

- Validation côté client : les formulaires sont contrôlés en JavaScript avant envoi (champs obligatoires, format de l'email, longueur du mot de passe).
- Validation côté serveur : une seconde vérification est effectuée en PHP (fonction `validateRequired`, `filter_var`) afin de renforcer la sécurité et garantir l'intégrité des données.
- Gestion des erreurs : en cas de données invalides, des messages d'erreur explicites (code HTTP 422 et description par champ) sont renvoyés par `errorResponse()` et affichés à l'utilisateur.

5. Tests de la base de données

Les tests réalisés ont permis de vérifier :

- L'insertion correcte des données dans les différentes tables MySQL ;
- La génération unique des identifiants publics (UUID) pour les transactions, utilisateurs et abonnements ;
- La mise à jour correcte des informations enregistrées (statut des transactions, profils utilisateurs) ;
- Le bon fonctionnement des opérations CRUD (Create, Read, Update, Delete) sur les plans et les coupons ;
- Le respect des relations entre les tables utilisateurs, abonnements et transactions.

Lors de ces tests, des colonnes manquantes ont été identifiées dans la table transactions (user_agent, billing_name, billing_email, billing_address) via les messages d'erreur PDO consignés dans storage/logs/php_errors.log, puis ajoutées via phpMyAdmin (instructions ALTER TABLE).

6. Tests d'interface utilisateur

- Compatibilité multi-supports : l'interface a été testée sur un poste de bureau ainsi qu'en mode responsive (réduction de la largeur de la fenêtre du navigateur).
- Compatibilité navigateur : vérification de l'affichage sur Google Chrome (navigateur principal utilisé lors des tests).
- Responsive design : la page de checkout adopte une mise en page en deux colonnes sur écran large et bascule en une seule colonne sur les petits écrans, grâce à une règle media query dans main.css.

7. Résultats

Les résultats des tests montrent que :

- L'application est fonctionnelle et stable ;
- Les principales fonctionnalités répondent aux besoins définis ;
- Les erreurs rencontrées lors des premiers tests (décrites ci-dessous) ont été corrigées ;
- L'expérience utilisateur est satisfaisante pour un parcours de paiement complet en mode test.

Anomalies identifiées et corrigées

Anomalie observe	Cause identifiée	Correction apportée
Erreur 401 persistante sur les routes /api/admin/*	Apache/XAMPP ne transmettait pas l'en-tete Authorization à PHP	Ajout d'une regle de reecriture dans .htaccess : RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
Erreur 500 sur POST /api/checkout	Colonnes manquantes dans la table transactions (user_agent, billing_name, billing_email, billing_address)	Ajout des colonnes manquantes via ALTER TABLE dans phpMyAdmin
Champ « numero de carte » Stripe non interactif	Montage de Stripe Elements avant le chargement complet du DOM	Initialisation de Stripe Elements dans l'evenement DOMContentLoaded
Avertissement « Undefined array key provider »	La cle provider n'etait pas transmise dans le tableau \$paymentData	Ajout explicite de 'provider' => \$provider dans CheckoutController
Champ Stripe Public Key vide cote client	Fichier .env contenant des cle placeholder (pk_test_xxxx)	Mise à jour du fichier .env avec les cle de test Stripe réelles

8. Validation du système

L'ensemble des essais démontre que Gestion des Paiements satisfait les besoins exprimés lors de l'analyse : authentification sécurisée, gestion fiable des abonnements et des coupons, traitement sécurisé des paiements via Stripe, administration complète et intégrité des données. A l'issue des phases de tests, l'application a été validée pour un environnement de développement/démonstration.

9. Conclusion

Les tests effectués ont confirmé que l'application est stable et fonctionnellement conforme pour le périmètre actuellement implémenté (fournisseur Stripe). Gestion des Paiements est prête à être présentée dans le cadre du projet de fin d'études, les fournisseurs PayPal restant à finaliser avant un éventuel passage en production.

VIII. Résultats et discussion

1. Introduction

Cette section présente une synthèse des résultats obtenus, une comparaison avec les objectifs initiaux du projet, ainsi qu'une discussion des apports, des difficultés rencontrées et des limites actuelles de la plateforme.

2. Resultats obtenus

- Une architecture MVC en PHP natif claire, organisée en contrôleurs, services, modèles et middlewares ;
- Un système d'authentification JWT fonctionnel avec rafraichissement automatique du jeton cote client ;
- Un parcours de paiement Stripe opérationnel de bout en bout (sélection du plan, récapitulatif avec TVA, saisie carte, confirmation) en environnement de test ;
- Un tableau de bord administrateur couvrant la supervision des transactions, utilisateurs, abonnements, plans, coupons et journaux ;
- Une base de données relationnelle cohérente couvrant l'ensemble du cycle de vie d'un paiement, y compris remboursements et factures.

3. Comparaison avec les objectifs initiaux

Les objectifs prioritaires (gestion du cycle de paiement, sécurité JWT, tableau de bord administrateur, gestion des plans et coupons, journalisation) ont été atteints. L'objectif d'intégration multi-fournisseurs est atteint partiellement : Stripe est pleinement opérationnel, tandis que PayPal,disposent d'une architecture prête (interface commune ProviderFactory, points d'entree webhook) mais nécessitent la configuration de clés d'API réelles et l'implémentation des appels spécifiques.

4. Apports du projet

- Mise en pratique d'une architecture MVC complète en PHP natif, sans framework ;
- Maitrise de l'authentification JWT (génération, vérification, rafraichissement) ;
- Intégration réelle d'une API de paiement tierce (Stripe) avec gestion sécurisée des cles via .env ;
- Expérience pratique de débogage via l'analyse de journaux d'erreurs PHP et de requetes SQL.

5. Difficulties rencontrees

- Configuration du serveur Apache/XAMPP pour la transmission de l'en-tête Authorization, nécessaire au fonctionnement du JWT ;
- Synchronisation entre le schéma de la base de données et les modèles PHP, ayant nécessité l'ajout de plusieurs colonnes à la table transactions ;
- Initialisation correcte des composants Stripe Eléments côté navigateur (timing du chargement du DOM) ;
- Gestion des clés d'API (placeholders vs clés de test réelles) dans le fichier de configuration .env.

6. Discussion et limits du projet

- Les fournisseurs PayPal ne sont pas encore opérationnels (message « bientôt disponible ») ;
- La génération automatique de factures au format PDF n'est pas encore implementee (la structure des données est prête via InvoiceService, la sortie PDF restant à développer) ;
- L'application n'a pas fait l'objet de tests de charge ni d'un déploiement en environnement de production ;
- Les clés utilisées (Stripe) sont des clés de test ; un passage en production nécessiterait des clés live et une revue de sécurité complémentaire.

7. Conclusion

Les résultats obtenus répondent globalement aux objectifs fixés pour ce projet de fin d'études, avec un périmètre fonctionnel solide autour de Stripe et de l'administration de la plateforme. Les limites identifiées constituent une base claire pour les perspectives d'évolution présentées ci-après.

IX. Conclusion générale et perspectives

1. Conclusion générale

Le travail présenté dans ce rapport met en lumière la conception et la mise en œuvre d'une plateforme de gestion de paiements en PHP natif, structurée selon une architecture MVC et sécurisée par authentification JWT. L'intégration de Stripe a été validée de bout en bout, depuis la sélection d'un plan jusqu'à la confirmation du paiement et la création de la transaction en base de données. Le tableau de bord administrateur offre une vue opérationnelle sur l'activité financière de la plateforme.

Au-delà de l'aspect technique, ce projet a constitué une mise en pratique concrète des compétences acquises durant la formation : conception de bases de données relationnelles, développement d'API sécurisées, intégration de services tiers et résolution méthodique d'anomalies par l'analyse de journaux applicatifs.

2. Perspectives d'évolution

- Finaliser l'intégration des fournisseurs PayPal, compris la vérification HMAC des webhooks ;
- Implémenter la génération automatique de factures au format PDF (InvoiceService) ;
- Conteneuriser l'application avec Docker pour faciliter le déploiement ;
- Etudier l'intégration de mécanismes de détection de la fraude (regles ou apprentissage automatique) ;
- Explorer l'ajout de paiements décentralisés (crypto-actifs) pour les transactions internationales.

X. Bibliographies

- PHP Documentation - The PHP Group, <https://www.php.net/docs.php>
- MySQL Reference Manual - Oracle Corporation, <https://dev.mysql.com/doc/>
- Stripe API - Stripe Inc., <https://stripe.com/docs/api>
- BCrypt - PHP Password-Hashing API, <https://www.php.net/manual/en/function.password-hash.php>
- Paypal; <https://www.paypal.com/fr/webapps/mpp/account-selection>
- github, <https://github.com/insafelgeurgar-jpg/PROTOTYP>
- figma, <https://www.figma.com/files/team/1569631982807832259/recents-and-sharing?fuid=1569631980543661271>